

# FORMAL LANGUAGES AND COMPILERS

prof.s Luca Breveglieri and Angelo Morzenti

**Exam of Wed 8 July 2015 - Part Theory**

NAME:

---

MATRICOLA:

SIGNATURE:

---

## INSTRUCTIONS - READ CAREFULLY:

- The exam is in written form and consists of two parts:
  1. Theory (80%): Syntax and Semantics of Languages
    - regular expressions and finite automata
    - free grammars and pushdown automata
    - syntax analysis and parsing methodologies
    - language translation and semantic analysis
  2. Lab (20%): Compiler Design by Flex and Bison
- To pass the exam, the candidate must succeed in both parts (theory and lab), in one call or more calls separately, but within one year (12 months) between the two parts.
- To pass part theory, the candidate must answer the mandatory (not optional) questions; notice that the full grade is achieved by answering the optional questions.
- The exam is open book: textbooks and personal notes are permitted.
- Please write in the free space left and if necessary continue on the back side of the sheet; do not attach new sheets and do not replace the existing ones.
- Time: part lab 60m - part theory 2h.15m

## 1 Regular Expressions and Finite Automata 20%

1. Consider the following regular expressions  $R_1$  and  $R_2$ , over the two-letter alphabet  $\Sigma = \{ a, b \}$ :

$$R_1 = (a a \mid b)^* \qquad R_2 = \Sigma^* a b \Sigma^*$$

Answer the following questions:

- (a) Draw the two deterministic automata  $A_1$  and  $A_2$  equivalent to the two regular expressions  $R_1$  and  $R_2$ , by using the Berry-Sethi method at least for  $A_2$ .
- (b) Check if the two automata  $A_1$  and  $A_2$  are minimal, and if necessary minimize them.
- (c) Find a deterministic automaton  $A_3$  equivalent to the regular expression  $R_3$  below, extended with the intersection operator and the complement one ( $\neg$ ):

$$R_3 = R_1 \cap \neg R_2$$

and if necessary clean the automaton  $A_3$  by removing the useless states.

- (d) (optional) Find, in a systematic way and in any case justifying your answer, a regular expression  $R_4$  that is equivalent to the extended regular expression  $R_3$  and that contains only the standard operators of concatenation, union and star (or cross).
-







## 2 Free Grammars and Pushdown Automata 20%

1. Answer the following questions:

(a) Write a grammar  $G_1$  for the language  $L_1$  below:

$$L_1 = \{ a^i b^j c^k \mid i \neq j \}$$

Write also a similar grammar  $G_2$  for the language  $L_2$  below:

$$L_2 = \{ a^i b^j c^k \mid j \neq k \}$$

(b) Based on the grammar  $G_1$  of language  $L_1$ , draw the syntax trees of the two strings:

$$a a b c \quad \text{and} \quad a b b$$

(c) (optional) Considering that the language  $L = \{ a^n b^n c^n \mid n \geq 1 \}$  is *not* context-free (i.e.,  $L \notin CF$ ), use the fact that  $L_1 \in CF$  and  $L_2 \in CF$  (as proved by your answer to question (a) above) to argue that the family of context-free languages  $CF$  is *not* closed under set complement.

---



2. Consider a data description language that can model the structure of an alphabetical text. The text is modeled by breaking it into substrings, which can be arranged into lists or arrays, and can be possibly given a name. Such a language has these features:

- A text is modeled as a (possibly empty) list of elements.
- A list is enclosed in graph brackets “{” and “}”, and the elements are separated by a comma “,”.
- An element has a name, which is an identifier schematized by the terminal `id`, followed by a colon “:” and then by a value.
- A value may be a string, an array or, recursively, a (possibly empty) list.
- A string is a non-empty sequence of letters (uppercase and lowercase), decimal digits and dots “.”.
- An array is a (non-empty) list of (unnamed) values; the array is enclosed in square brackets “[” and “]”, and the values are separated by a semicolon “;”.

Example: the following text

```
xy.zAabc..0121alphaAX0
```

is modeled by the data description language as:

```
{
  one : xy.zA,
  two : [
    a; bc..0; 12
  ],
  next : {
    inner : [
      1; alpha
    ]
  },
  last : AX0
}
```

Write a grammar, in the extended form (*EBNF*) and not ambiguous, that generates such a data description language.

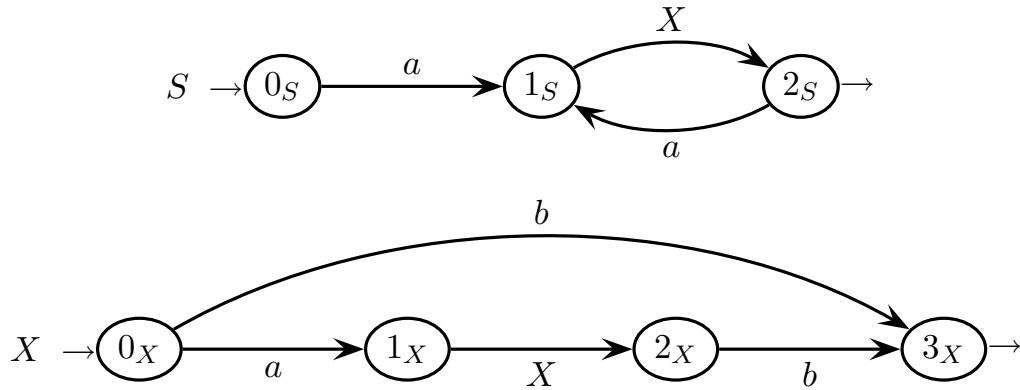
---





### 3 Syntax Analysis and Parsing Methodologies 20%

1. Consider the grammar  $G$  below, modeled as a net of recursive machines (axiom  $S$ ):



Answer the following questions:

- (a) Draw the complete *ELR* pilot of grammar  $G$ , say if the grammar is of *ELR*(1) type and justify your answer.
- (b) Examine the *ELR* pilot of grammar  $G$ , say if the grammar is of *ELL*(1) type and justify your answer. Furthermore, compute all the guide sets (on all the arcs and the exit arrows).
- (c) Simulate the *ELR* (shift-reduce) analysis of the sample string below:

$a a b b a b$

which belongs to the language  $L(G)$ . Use the table prepared on the next pages.

- (d) (optional) Simulate the *ELL* (predictive) analysis of the sample string below:

$a a b b a b$

which belongs to the language  $L(G)$ . Use the table prepared on the next pages.

simulation table for the *ELR* analysis to be completed  
(the number of rows is not significant -  $I_0, I_1, \dots$  are the pilot macro-states)

[illegible]

simulation table for the *ELL* analysis to be completed  
(the number of rows is not significant)

[illegible]







## 4 Language Translation and Semantic Analysis 20%

1. Consider a language  $L$  of strings that are two-level lists (non-empty). An atomic element of a list is schematized by the terminal  $e$ . Any two consecutive elements (atomic or not) are separated by a comma “,”. The lists at the second level are enclosed in round brackets. Here are a few sample lists:

$e \quad e, e \quad (e) \quad e, (e, e) \quad e, e, (e, e), e \quad (e), (e)$

Answer the following questions:

- (a) Consider a syntactic translation  $\tau_1$  of the language  $L$ , which flattens the list by replacing every second-level list with a terminal placeholder “ $\perp$ ”. For instance:

$$\tau_1(e, e, (e, e), e) = e, e, \perp, e$$

Write a syntactic translation scheme equivalent to such a translation  $\tau_1$ . Use the source grammar  $G_s$  below, which generates the language  $L$ , and complete it with the destination grammar  $G_d$ .

- (b) (optional) Consider a syntactic translation  $\tau_2$  of the language  $L$ , which swaps the first-level and second-level elements. For instance:

$$\tau_2(e, e, e, (e, e), e) = (e, e, e), e, e, (e)$$

Write a syntactic translation scheme equivalent to such a translation  $\tau_2$ . If necessary, you can change the source grammar.

*source  $G_s$  for (a) (axiom  $L_1$ )*

*destination  $G_d$  for (a)*

$$L_1 \rightarrow e, L_1$$

$$L_1 \rightarrow e$$

$$L_1 \rightarrow (L_2), L_1$$

$$L_1 \rightarrow (L_2)$$

$$L_2 \rightarrow e, L_2 \mid e$$





2. Lists of nested sequences of dots, such as  $(((.)))((.))$ , are used to encode positive integer numbers as sums of powers. For instance, this list:

$$\underbrace{\left( \left( \left( \overset{\text{2 dots}}{\underbrace{\quad\quad}_{\cdot\cdot}} \right) \right) \right)}_{\text{depth 3}} \quad \underbrace{\left( \left( \overset{\text{1 dot}}{\underbrace{\quad\quad}_{\cdot}} \right) \right)}_{\text{depth 2}}$$

encodes the number  $3^2 + 2^1 = 9 + 2 = 11$ , while the list  $((\dots))(((\dots)))$  encodes  $2^5 + 4^3 = 32 + 64 = 96$ . Hence the number of dots in a sequence represents the exponent, and its nesting depth the base of each power in the sum.

The following grammar generates the list (axiom  $L$ ):

$$\left\{ \begin{array}{l} L \rightarrow E L \mid E \\ E \rightarrow ( E ) \mid ( P ) \\ P \rightarrow . P \mid . \end{array} \right.$$

- (a) Write an attribute grammar that computes the value of the number encoded by the list. The final value is associated to the tree root. It is suggested to use the following attributes:
  - $d$  is the nesting level of each sublist
  - $n$  is the number of dots in each sublist
  - $v$  is the numerical value encoded
- (b) Decorate the syntax tree of the sentence  $((((\cdot)))((\cdot)))$ , by adding to the tree the values of all the attributes. Use the tree prepared on the next page.
- (c) (optional) Determine whether the attribute grammar written is one-sweep and adequately justify your answer.

attributes assigned to be used (complete the specification)

| <i>type</i> | <i>name</i> | <i>domain</i> | <i>nonterm.</i> | <i>meaning</i>                 |
|-------------|-------------|---------------|-----------------|--------------------------------|
|             | <i>d</i>    | integer       |                 | nesting level of each sublist  |
|             | <i>n</i>    | integer       |                 | number of dots in each sublist |
|             | <i>v</i>    | integer       |                 | numerical value encoded        |

attributes to be added, if any

| <i>type</i> | <i>name</i> | <i>domain</i> | <i>nonterm.</i> | <i>meaning</i> |
|-------------|-------------|---------------|-----------------|----------------|
|             |             |               |                 |                |

| # | <i>syntax</i> | <i>semantics</i> |
|---|---------------|------------------|
|---|---------------|------------------|

---

1:  $L_0 \rightarrow E_1 L_2$

---

2:  $L_0 \rightarrow E_1$

---

3:  $E_0 \rightarrow ( E_1 )$

---

4:  $E_0 \rightarrow ( P_1 )$

---

5:  $P_0 \rightarrow . P_1$

---

6:  $P_0 \rightarrow .$

syntax tree to be decorated

